

Physics-Informed Neural Operators for Cardiac Electrophysiology

Hannah Lydon

HANNAH.LYDON@KCL.AC.UK

Milad Kazemi

MILAD.KAZEMI@KCL.AC.UK

Department of Informatics, King's College London, UK

Martin Bishop

MARTIN.BISHOP@KCL.AC.UK

Research Department of Digital Twins in Healthcare, King's College London, UK

Nicola Paoletti

NICOLA.PAOLETTI@KCL.AC.UK

Department of Informatics, King's College London, UK

Editors: G. Sukhatme, L. Lindemann, S. Tu, A. Wierman, N. Atanasov

Abstract

Accurately simulating systems governed by PDEs, such as voltage fields in cardiac electrophysiology (EP) modelling, remains a significant modelling challenge. Traditional numerical solvers are computationally expensive and sensitive to discretisation, while canonical deep learning methods are data-hungry and struggle with chaotic dynamics and long-term predictions. Physics-Informed Neural Networks (PINNs) mitigate some of these issues by incorporating physical constraints in the learning process, yet they remain limited by mesh resolution and long-term predictive stability. In this work, we propose a *Physics-Informed Neural Operator (PINO)* approach to solve PDE problems in cardiac EP. Unlike PINNs, PINO models learn mappings between function spaces, allowing them to generalise to multiple mesh resolutions and initial conditions. Our results show that PINO models can accurately reproduce cardiac EP dynamics over extended time horizons and across multiple propagation scenarios, including zero-shot evaluations on scenarios unseen during training. Additionally, our PINO models maintain high predictive quality in long roll-outs (where predictions are recursively fed back as inputs), and can scale their predictive resolution by up to $10\times$ the training resolution. These advantages come with a significant reduction in simulation time compared to numerical PDE solvers, highlighting the potential of PINO-based approaches for efficient and scalable cardiac EP simulations. All code used in this work, including experimental results, can be found at <https://github.com/janet-9/CardiacEP-PINOS>

Keywords: Neural Operators, PDEs, Predictive modelling

1. Introduction

Modelling the behaviour of complex dynamical systems governed by partial differential equations (PDEs) is a common problem in engineering, particularly for simulating physical phenomena such as fluid flow and reaction-diffusion systems. Many biological processes can be modelled by PDE systems, and this framing has led to a growth of computational models that aim to simulate the human body to help guide diagnosis and treatment options (Katsoulakis et al.).

In cardiac healthcare, modelling electrophysiological dynamics poses a key challenge for understanding the origins of potentially dangerous arrhythmias and can aid in planning life-saving treatments, such as catheter ablation or the implantation of cardiac devices. To this purpose, several approaches have been explored to simulate cardiac electrophysiology (EP), e.g. (Jiang et al., 2016; Trayanova et al., 2024; Lydon et al., 2025). These methods are often based on mechanistic modelling, such as finite element simulations (Plank et al., 2021). As such, they tend to be highly sensitive to mesh resolution

and model parameter selection, often resulting in trade-offs between maintaining model accuracy and ensuring computationally efficient simulations.

Building on the advances and successes of deep learning, learning-based methods have been recently applied to PDE modelling as well, resulting in accurate approximations of the PDE dynamics at a fraction of the simulation/inference cost. Physics-informed neural networks (PINNs) (Raissi et al., 2019) are a popular technique, which combines a usual data loss term with a physics-based loss to penalise deviations from the (known) laws governing the system. In fact, PINNs have already been explored in the field of cardiac EP to model activation sequences in cardiac tissue (Sahli Costabal et al., 2020), as well as more complex cardiac dynamics in both 2D and 3D geometries (Herrero Martin et al., 2022; Chiu et al., 2024; Nazarov et al., 2022; Werneck et al., 2023; Zolotarev et al., 2025). Whilst PINNs have shown significant promise in replicating the accuracy of traditional finite-element EP models, they are limited to simulating on a fixed mesh resolution and often require extensive training budgets to capture the features of the PDE system. Additionally, because PINN models serve as function approximators for a single PDE instance, they require re-training to adapt to unseen model parameters, which limits their generalisation capabilities.

More recent *Neural Operator* (NO) models (Kossaifi et al., 2024; Kovachki et al., 2021) address the above limitations by shifting the training paradigm from fitting a network for one fixed PDE instance (the PINN approach) to learning mappings – or, operators – between function spaces, where the input function normally represents an initial condition and the output represents the corresponding PDE solution. As such, NO models can learn the dynamics of families of PDEs rather than a single instance (making them independent of the initial condition), can scale efficiently to high dimensions, and are resolution-invariant, i.e., they support inference at different (unseen) discretisation levels. Similar to the development of PINNs, NO models have been extended to incorporate physics losses to enforce agreement with known dynamics, an approach called *Physics-Informed Neural Operators* (PINOs) (Li et al., 2024).

So far, (PI)NO models have primarily been applied to canonical PDE problems such as fluid equations (e.g., Burger’s equation and Navier-Stokes model). However, their ability to predict across a family of parametric PDEs (at a fraction of the cost of the original PDE) and generalise across resolutions (akin to numerical methods) makes them an ideal candidate for tackling EP modelling scenarios, which is the main objective of our work.

Contributions: This work presents the first physics informed neural operator-based method for the simulation of cardiac electrophysiology scenarios, incorporating known cell models as soft physics constraints with a Fourier neural operator (FNO) (Li et al., 2020) backbone. Through extensive experiments, we demonstrate that our approach can:

- Produce model predictions of equivalent accuracy to numerical simulations for a range of cardiac wave propagation scenarios.
- Perform predictions on unseen data with resolutions significantly higher than those available during training, demonstrating the flexibility of our model.
- Achieve high-quality predictions in long roll-out simulations for unseen time horizons. That is, our PINO model also performs well when predictions are made sequentially from past predictions rather than the true inputs, demonstrating its applicability to online prediction scenarios where ground-truth data may be infrequent.

2. Background

2.1. Neural Operator Model

We define our general PDE system in the bounded domain $\mathcal{D}$ as

$$\frac{\partial u}{\partial t} = \mathcal{R}(u) \text{ on } \mathcal{D} \times (0, \infty), \quad u = g \text{ on } \partial\mathcal{D} \times (0, \infty), \quad u = a \text{ on } \overline{\mathcal{D}} \times \{0\}, \quad (1)$$

where we have a partial differential operator $\mathcal{R}$, a known initial condition $a = u(0)$ (where $\overline{\mathcal{D}}$ is the closure of $\mathcal{D}$, all spatial points including the boundary at time $t = 0$), and a known boundary condition g . For operator learning, we consider a family of PDE models described by Equation 1 with Banach spaces $\mathcal{A}$ (the space of initial condition functions) and $\mathcal{U}$ (the space of solution functions). For our initial and unknown conditions, we let $a = u(0) \in \mathcal{A}$ and $u(t) \in \mathcal{U}$ for $t > 0$.

Using this setting, we aim to learn $\mathcal{R}$ using a neural operator model in the method of (Kossaifi et al., 2024; Kovachki et al., 2021). We take the solution operator to be $\mathcal{G}^\dagger : \mathcal{A} \rightarrow \mathcal{U}$ induced by the mapping $a \rightarrow u$. The solution operator $\mathcal{G}^\dagger$ can be approximated by the neural operator model represented below:

$$\mathcal{G}_\theta = \mathcal{Q} \circ \sigma(\mathcal{W}_L + \mathcal{K}_L + b_L) \circ \dots \circ \sigma(\mathcal{W}_1 + \mathcal{K}_1 + b_1) \circ \mathcal{P}, \quad (2)$$

which consists of two point wise operators $\mathcal{P} : \mathbb{R}^{d_a} \rightarrow \mathbb{R}^{d_L}$ and $\mathcal{Q} : \mathbb{R}^{d_L} \rightarrow \mathbb{R}^{d_u}$ such that $\mathcal{P}$ lifts the input function $a \in \mathcal{A}$ from dimension d_a to dimension d_L of the hidden layers, and $\mathcal{Q}$ projects the output of the hidden layers to dimension d_u of the unknown function $u \in \mathcal{U}$. The main section of the network consists of L layers of a parametrised pointwise linear operator $\mathcal{W}_l$ (a weight matrix), a constant bias b_l , and a parametrised kernel operator $\mathcal{K}_l$, which are combined and processed by a fixed activation function σ . All of the parameters of the network are described by θ . The kernel function $\mathcal{K}_l$ can take various forms depending on the task. In our case, we consider $\mathcal{K}_l$ to be the Fourier convolution operator, resulting in a Fourier Neural Operator model (FNO) as described by (Li et al., 2020). This kernel operator is described by Equation 3 below

$$(\mathcal{K}v^l)(x) = \mathcal{F}^{-1}(R \cdot (\mathcal{F}v^l))(x) \quad (3)$$

where $\mathcal{F}, \mathcal{F}^{-1}$ are the fast Fourier transform and its inverse, v^l is the function representation under the lifting operator P in layer l of the model, and R represents a matrix of learnable weights. This kernel function can be viewed as a convolution operating in Fourier space, where R assigns different weights to each frequency component. The motivation for employing this kernel is that by performing parametrisation in the Fourier domain we can reduce the complexity of the kernel calculations whilst still learning global features, which is ideal for PDE modelling¹.

2.2. Physics-informed Learning

A major challenge in applying machine learning to solve PDE problems is that enforcing physical constraints solely from data is difficult and may require large amounts of data, which can be prohibitive for these systems. The field of physics-informed machine learning addresses this problem by incorporating domain knowledge to guide the model construction. Physics-Informed Neural Networks (PINNs) (Raissi et al., 2019) are one of the first and most prominent methods. PINNs incorporate

1. To use the fast Fourier transforms, the functions are assumed to be on a uniform and discrete mesh. If not, then an additional transformation (typically a graph kernel operator) can be applied to map the functions to a uniform mesh.

physics knowledge as a soft constraint on the neural network by creating loss functions that penalise deviations from the PDE residuals as well as any known boundary and initial conditions.

A general form for this loss function, using ℓ_2 -norm residuals, can be seen in Equation 4, where u_θ denotes the solution function found by the network, a is a given initial condition, $\mathcal{R}$ is the differential operator, and λ_{res} , λ_{bc} , λ_{ic} are weights for each physics loss component².

$$\begin{aligned} \mathcal{L}_{\text{phys}}(a, u_\theta) = & \mathcal{L}_{res} + \mathcal{L}_{IC} + \mathcal{L}_{BC} = \lambda_{res} \int_0^T \int_D \left\| \frac{du_\theta}{dt}(t, x) - \mathcal{R}(u_\theta(t, x)) \right\|_2 dx dt \\ & + \lambda_{bc} \int_0^T \int_{\partial D} \|u_\theta(t, x) - g(t, x)\|_2 dx dt + \lambda_{ic} \int_D \|u_\theta(0, x) - a(x)\|_2 dx \end{aligned} \quad (4)$$

As discussed in the Introduction, PINNs can only learn a single PDE instance at a time, requiring retraining with a new instance for each solution function $u \in \mathcal{U}$. Moreover, PINNs rely on discretised meshes to approximate the derivatives required in the PDE residual loss. This requires evaluating the network and its gradients at a large number of collocation points, which scales poorly with both the domain's dimensionality and the complexity of the PDE. Finally, due to their nature as an iterative solver, PINNs can struggle to generalise beyond the spatial and temporal domains seen during training.

These limitations can be addressed by using operator learning (see Section 2.1), which, by working over function spaces, can learn a family of PDE instances and generalise across resolutions in a data-efficient manner. We next formulate data and physics losses in the NO context. Let $\mathcal{G}_\theta$ be the operator model of Equation 2, learned from a dataset $\{a_j, u_j = \mathcal{G}^\dagger(a_j)\}_{j=1}^N$, where $\mathcal{G}^\dagger$ is the true solution operator we seek to approximate. We assume the dataset is induced by a distribution μ of initial conditions of interest. For a given function pair (a, u) , the data loss for $\mathcal{G}_\theta$ is

$$\mathcal{L}_{\text{data}}(a, u) = \int_D \|u(x) - \mathcal{G}_\theta(a)(x)\|_2 dx \quad (5)$$

where the integrals are calculated as the weighted sum of all available spatial points³.

Similarly to PINNs, the final loss is obtained by combining $\mathcal{L}_{\text{data}}$ and $\mathcal{L}_{\text{phys}}$ (see Equation 4) via weighting parameters λ_{data} and λ_{phys} , and averaging over the dataset:

$$\begin{aligned} \mathcal{L} = & \mathbb{E}_{a \sim \mu} \left[\lambda_{\text{data}} \mathcal{L}_{\text{data}}(a, \mathcal{G}^\dagger(a)) + \lambda_{\text{phys}} \mathcal{L}_{\text{phys}}(a, \mathcal{G}_\theta(a)) \right] \approx \\ & \frac{1}{N} \left[\sum_{j=1}^N \lambda_{\text{data}} \mathcal{L}_{\text{data}}(a_j, u_j) + \lambda_{\text{phys}} \mathcal{L}_{\text{phys}}(a_j, \mathcal{G}_\theta(a_j)) \right]. \end{aligned} \quad (6)$$

3. Problem Setting and Model Design

3.1. Problem Setting

We develop a physics-informed neural operator (PINO) approach for PDE-based cardiac electrophysiology. Previous work have considered this class of systems (Herrero Martin et al., 2022; Chiu et al.,

2. Both the weightings of the loss components and the parameters of $\mathcal{R}$ can either be fixed or learnt as part of the network parameters, θ .

3. In the case that the functions are structured as grids, these weights are equal.

2024; Zolotarev et al., 2025; Nazarov et al., 2022) but using a PINN approach. By leveraging PINOs, we aim to provide a more flexible model able to 1) reproduce different cardiac propagation scenarios without having to retrain the model for each instance and 2) adapt to varying mesh resolutions, thereby drastically reducing both training and runtime costs.

The specific PDE problem we consider concerns simulating the propagation of voltage fields across 2D meshes using the Aliev-Panfilov cell model (Aliev and Panfilov, 1996). This model captures the shape of cardiac action potentials using a reaction diffusion system represented by the following pair of coupled PDEs:

$$\frac{\partial V}{\partial t} = \nabla \cdot (D \nabla V) - kV(V-a)(V-1) - VW \quad (7)$$

$$\frac{\partial W}{\partial t} = \left(\epsilon + \frac{\mu_1 W}{V + \mu_2} \right) [-W - kV(V-a-1)], \quad (8)$$

where V represents the transmembrane voltage in scaled dimensionless units, W represents a latent recovery variable and t represents a scaled dimensionless time variable. Parameters k and ϵ represent rate constants for excitation and restitution respectively, a^4 represents a threshold parameter for excitation and μ_1 and μ_2 are scalable parameters that control the shape of the action potential curve. In particular, a and μ_1 can be altered to induce chaotic dynamics in the system, as was demonstrated in (Panfilov, 2002). In our model we consider the domain to be isotropic, which allows the diffusion tensor D to be a scalar value and therefore reduces the first term in Equation 7 to the Laplacian $D \nabla^2 V$. Further details on the model implementation, including parameter selection and unit scaling information, can be found in Section S1.

3.2. Model Design

Model architecture Our model uses a Fourier Neural Operator (FNO) backbone (see Equations 2 and 3) to capture the system’s global dynamics. The model architecture, as described in Section 2.1, consists of an initial grid embedding layer that encodes the positional information of the field maps, a lifting layer to expand to higher dimensions, four FNO blocks consisting of a Fourier convolution kernel, a combination of skip connections and channel MLPs with soft gating modulation, and a projection layer to map the field back to the original domain.

Data generation The training data consists of 2D spatio-temporal trajectories of transmembrane voltage V and recovery current W fields (the two variables of our PDE). All samples were processed in batches of dimensionality $[B, D, L, H_{grid}, W_{grid}]$, with B being the batch size, D the field dimension⁵, L the trajectory length and $H_{grid} \times W_{grid}$ the (spatial) training resolution. For data generation, the training time horizon $[0, T]$ – disjoint from the test horizon – was discretised using a regular grid, i.e., $0 = t_1, \dots, t_k = T$. We considered two dataset formulations:

- *Single Frame Training* ($L = 1$): the model is given a single time frame as input and is trained to predict the time frame at n steps ahead. Specifically, for time point $i = 1, \dots, k - n$, we associate a data point (a, u) where a is the PDE solution at t_i , i.e., $a = u^\dagger(t_i)$, and u is the solution after n steps, i.e., $u = u^\dagger(t_{i+n})$.

4. Not to be confused, in this context, with the operator input a (a voltage map).

5. In our case, the voltage and recovery fields respectively where $d = 0$ is the voltage field and $d = 1$ is the recovery field.

- *Multi Frame training* ($L = m$): here, the model receives and predicts trajectories of length m instead of individual frames. In this case, for $i = 1, \dots, k - n - 2m$, we define the input a as the trajectory $a = (u^\dagger(t_i), \dots, u^\dagger(t_{i+m}))$ and its outcome as $u = (u^\dagger(t_{i+m+n}), \dots, u^\dagger(t_{i+2m+n}))$.

The reason we consider these two different formulations is to assess whether the predictive quality of the model could be improved by training with small sequences that evolve over time as opposed to single time snapshots, particularly when evaluating the model in sequential roll-outs (as described in our experiments of Section 4.1).

Physics loss computation We note that the choice of frames does not affect the data loss term, as this is defined over the spatial domain only (see Equation 5), but it does affect the physics-informed loss $\mathcal{L}_{\text{phys}}$ and specifically, the PDE residual term (Equation 4), as this requires temporal information to approximate the derivatives. In particular, the residual in Equation 5 is approximated using the central difference method, i.e., for time t_i and spatial point x :

$$\frac{du_\theta}{dt}(t_i, x) - \mathcal{R}(u_\theta(t_i, x)) \approx \left(\frac{u_\theta(t_{i+1}) - u_\theta(t_{i-1})}{t_{i+1} - t_{i-1}} \right) - \mathcal{R}(u_\theta(t_i, x)) \quad (9)$$

where $u_\theta(t_i)$ is the model prediction at time t_i , and the term $\mathcal{R}_\Theta(u_\theta(t_i, x))$ is obtained by plugging the field $u_\theta(t_i, x)$ into the RHS of the PDE equations 7 and 8.

The way Equation 9 is computed depends on the choice of frame structure. As described in (Li et al., 2024), in the multi-frame approach, the time derivative is calculated within the trajectory, i.e., for sample b in the batch, field dimension d , and spatial point (h, w) , the difference $u_\theta(t_{i+1}) - u_\theta(t_{i-1})$ is obtained as the prediction indexed by $[b, d, t_{i+1}, h, w]$ minus that indexed by $[b, d, t_{i-1}, h, w]$. In the single-frame setting, where we have individual points instead of trajectories, the derivative is calculated between consecutive samples of the batch, ensuring first that samples are organised sequentially within the batch: in this case, the above difference is obtained as the prediction indexed by $[t_{i+1}, d, 1, h, w]$ minus that indexed by $[t_{i-1}, d, 1, h, w]$.

The residual loss for each of Equations 7 and 8 are combined via a weighted sum, with the weights for each residual being scaled in order to bias capturing either the voltage or recovery field. The boundary conditions in both training methods are modelled as Neumann (no flux) boundary conditions that are enforced on the mesh borders of the output prediction⁶, and the initial condition loss is constructed for the multi-frame approach in order to minimise the errors between the first frame of the ground truth and predicted sample trajectories - that is, minimising the difference between $u^\dagger(t_0)$ and $u_\theta(t_0)$ to ensure consistency with the ground truth at the start of the predicted trajectory⁷.

Training algorithm Training proceeds in two stages: the first stage optimises only the data loss, and is followed by a second stage where both data and physics losses are applied. In particular, we explore three solutions to integrate the data and physics loss components during the second stage:

- *Fixed weights*: λ_{data} and λ_{phys} remain constant.
- *Soft adaptation* (Heydari et al., 2019): λ_{data} and λ_{phys} are adapted based on their relative magnitudes and rates of change.

6. This condition, the typical boundary condition used for cardiac electrophysiology modelling, is used to comply with the boundary conditions used in the differential solver that was used to generate the training data, and differs from the general boundary condition in (1). We note that during training, the data is padded at the borders to comply with the assumed periodicity in the FNO.

7. This loss term is not included for the single frame training.

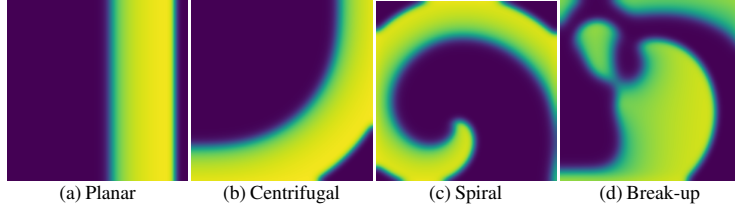


Figure 1: Propagation scenarios used for training the operator model, all sampled at $t = 500$ ms.

- *Relative aggregation* (Bischof and Kraus, 2025): λ_{data} and λ_{phys} are scaled relative to their previous values, but are occasionally reset to their initial values to avoid bias toward recent fluctuations.

Inference At inference time, we evaluate the prediction quality using only the data loss (called RMSE in the experimental section). We consider two evaluation settings:

- *Point-to-point (P2P)*, where the model receives the ground truth input for prediction. Here, this is equivalent to a single time-step prediction for the single-frame model and a prediction of multiple consecutive time-steps for the multi-frame model.
- *Roll-out*, where the model uses past predictions (rather than ground truth) to make new predictions. This setting is obviously more challenging, but serves as a test bed to assess whether our PINO model can be used as a simulator that operates autonomously without true data.

4. Experiments

We consider four propagation scenarios for our operator-based simulation: planar, centrifugal (propagation from a single corner of the mesh), spiral and spiral-breakup (pictured in Figure 1). The first two scenarios represent typical propagation patterns in healthy heart scenarios, whilst the latter are characteristic of arrhythmic conditions (with the spiral and spiral-breakup being analogous to monomorphic and polymorphic tachycardia/fibrillation, respectively). Training and evaluation data for all of the propagation scenarios were generated using the open-source cardiac modelling software package openCARP (Plank et al., 2021), a high-fidelity PDE simulator for cardiac electrophysiology. Further details of the PDE simulations for data generation can be found in Section S1.

To create the training sets we sampled the trajectories of 1000ms with a temporal resolution of $dt = 5$ ms and created the input and output pairs for both single and multi-frame input-output pairs using a moving time window. For the majority of the experiments, we then split the full set of function pairs into a training set representing the first 80% of the trajectory and used the rest of the trajectory as the testing set to encourage the model to extrapolate beyond the training time horizon⁸. For the multi-frame training, we used trajectories of length $m = 5$ (corresponding to 20-ms trajectories, or $\sim 2.0\%$ of the trajectory).

We trained all of the models using an Adam optimiser with a cosine learning rate scheduler (using an initial learning rate of 5×10^{-4} and a smooth decay over 1000 epochs). On a GPU-enabled server mounting an NVIDIA A40 48GB, the training time took an average of ≈ 1 hour for single-frame training and ≈ 7 hours for multi-frame training.

⁸. We added a buffer of $2n - 1$ frames between the testing and training sets in order to avoid data leakage at the border.

Table 1: Baseline comparison for the best performing FNO and PINO models for a fixed mesh resolution.

Scenario	Model	Single Frame		Multi Frame	
		RMSE (P2P)	RMSE (Roll-Out)	RMSE (P2P)	RMSE (Roll-Out)
Planar	FNO	0.3766	0.3819	0.3752	0.3794
	PINO	0.0095	0.2561	0.0195	0.0204
	$\Delta(\text{FNO-PINO})$	0.3671	0.1258	0.3557	0.3590
Centrifugal	FNO	0.0466	0.3826	0.0629	0.3747
	PINO	0.0178	0.3753	0.0057	0.0112
	$\Delta(\text{FNO-PINO})$	0.0288	0.0073	0.0572	0.3635
Spiral	FNO	0.0248	0.4554	0.0507	0.1682
	PINO	0.0047	0.0047	0.0113	0.0173
	$\Delta(\text{FNO-PINO})$	0.0201	0.4507	0.0394	0.1509
Spiral-Break	FNO	0.0514	0.5939	0.1144	0.3132
	PINO	0.0197	0.3961	0.0758	0.1925
	$\Delta(\text{FNO-PINO})$	0.0317	0.1978	0.0386	0.1207

4.1. Baseline Evaluations

The first experiment we performed was to evaluate the PINO model described in Section 3 under baseline conditions, (using a fixed mesh resolution for both training and evaluation). The best performing model of the initial training phase, using only data driven losses, was used as the FNO baseline and as the initial state for the secondary training round that incorporates physics losses. The secondary training phases ran for 1000 epochs (details of variations in training epochs can be found in Section S2.3) and, as explained in Section 3, we consider three PINO variants based on whether and how the physics loss weights are updated during training⁹. Each model was trained in both single-frame and multi-frame modes; and evaluated in a point-to-point (P2P) or roll-out (i.e., sequential) setting.¹⁰ The results are displayed in Table 1, comparing the purely data driven model to the best performing PINO model out of the three training strategies, and in Figures 2 and 3. The full table, as well as additional baseline evaluations in which we compare the FNO and PINO model against a PINN, can be found in Section S2.1.

The results demonstrate that the PINO approach shows either an equivalent or improved performance compared to the FNO model, with an especially strong improvement for the multi-frame roll-out evaluation, which is the most challenging evaluation scenario. This suggests that the inclusion of physics losses improves the model’s ability to make predictions over long time horizons without consistent access to ground truth inputs.

In addition to the results in Table 1 We additionally tested the available models on longer trajectories for the spiral and chaotic datasets, amounting to a full simulation trajectory of 2500 ms. These results can be found in Section S2.2 and Figure 2. At inference time, the trained model was able to predict a full trajectory via the roll-out method in an average of approximately 1s for all propagation types.

9. For the fixed weights PINO model, the losses for the secondary training phase were set as $\lambda_{\text{res}}, \lambda_{\text{ic}}, \lambda_{\text{bc}} = 0.01, 0.1, 0.1$.

10. We note we evaluated models on the predictions of the transmembrane voltage field only since the recovery field is a latent variable.

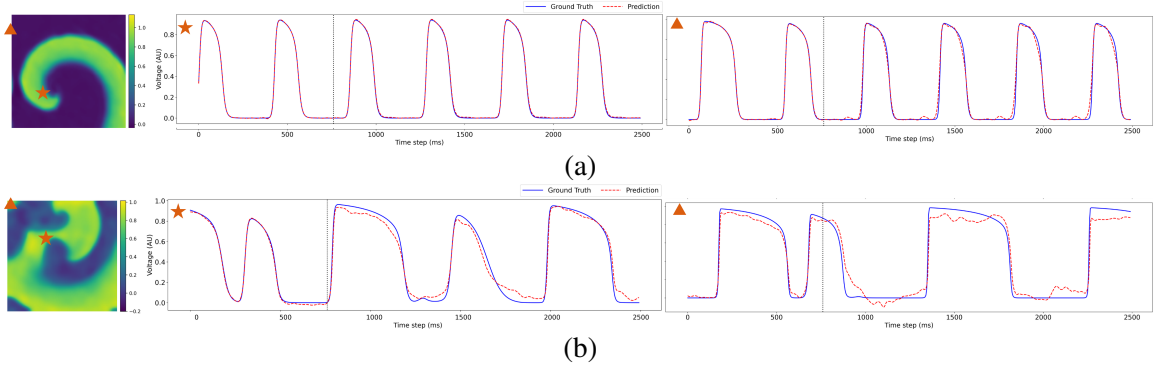


Figure 2: Point-to-point long-time horizon predictions for the (a) spiral and (b) spiral break-up propagation scenarios. The vertical dashed line represents the time horizon for the training data. The best (star) and worst (triangle) performing cells are marked on the voltage maps. The sample snapshots were taken at $t=1700\text{ms}$.

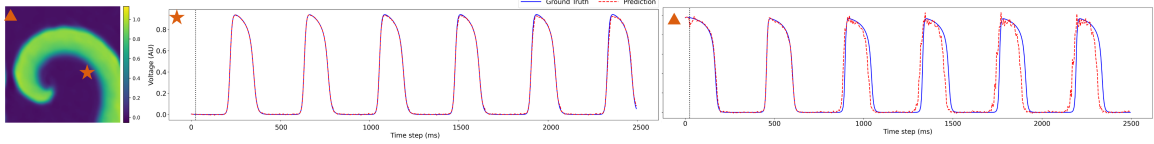


Figure 3: Results using the multi-frame roll-out evaluation method for the spiral scenario. The dashed line represents the initial time window given as an input to the model and the best (star) and worst (triangle) performing cells are marked on the sample field, taken at $t = 1250\text{ ms}$.

4.2. Mesh Resolution Testing

In order to evaluate the ability of the operator model to extrapolate between different mesh resolutions at inference time, we trained the best-performing PINO model (from the baseline tests) for each scenario on a mesh that was downsampled in spatial resolution by a factor of 10 from the mesh used to generate the ground truth simulation data. We then evaluated it on target meshes (using P2P evaluation) with resolutions scaled up relative to the training mesh by factors of 1.25, 2.50, 5.00, and 10.00. As shown in Figure 4, even when increasing the mesh resolution by a factor of 10 relative to the training data, the decrease in model accuracy is $< 8\%$ across all propagation scenarios.

4.3. Zero-Shot Transfer

Finally, we evaluated the performance of our operator model in a zero-shot transfer scenario – i.e., on datasets that were completely unseen during training – to test the model’s ability to learn the system’s underlying dynamics and use that knowledge to predict the trajectory of an unseen propagation scenario. For example, Figure 5 shows the best-performing models trained exclusively on the planar and centrifugal scenarios, evaluated on the (stable) spiral dataset. Despite being trained on simple scenarios, both models can accurately predict the patterns of a more complex propagation scenario. The only difference in the model predictions is in the scaling of the voltage field amplitudes, while the qualitative dynamics remain faithful. Further exploration of the model’s generalisation capabilities,

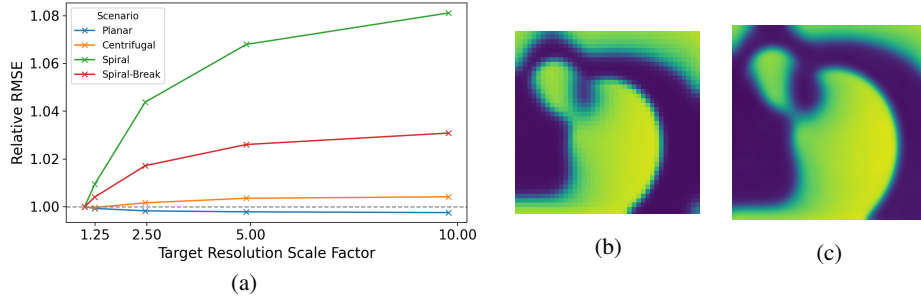


Figure 4: Zero-shot mesh resolution tests. (a) Relative RMSE values for P2P evaluation on increasing target resolution scales. (b) Input for the chaotic scenario at training resolution ($t = 495$ ms). (c) Prediction at the highest evaluation resolution ($t = 500$ ms).

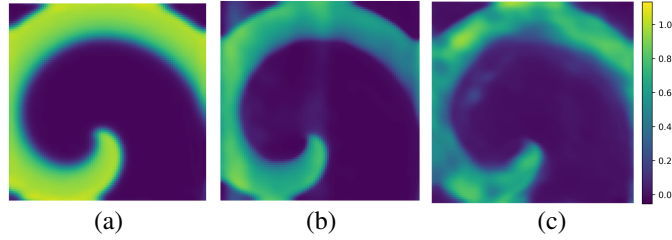


Figure 5: Zero-shot transfer. Samples shown are the predictions for the stable spiral scenario at $t = 500$ ms using the (a) Stable spiral model. (b) Planar model. (c) Centrifugal model.

demonstrating the ability to learn across varied parametrisation of the cell model and further examples of zero-shot transfer for propagation scenarios can be found in Section S2.4.

5. Conclusion

In this work, we demonstrated that physics-informed neural operator (PINO) models can efficiently predict cardiac electrophysiology dynamics with comparable accuracy to numerical solvers, with the additional advantage of being able to predict across multiple propagation types and parametrisations of the PDE model without retraining. The model was able to perform well in both the single and multiple time step prediction scenarios and also showed promising results when evaluated as a recursive predictor, albeit trained using small datasets and modest training budgets. These results demonstrate the strong potential of PINO models for enabling accurate online predictions in resource-constrained medical devices.

Future work will focus on extending this framework to improve the stability of long horizon predictions and to train models using irregular and 3D meshes in order to reflect realistic cardiac anatomy, potentially through employing more recently developed geometry-aware and recurrent operator layers (Chen et al., 2025; Li et al., 2023; Liu-Schiaffini et al., 2026) into the model to capture more complex spatial patterns and temporal evolution. We also aim to develop a general operator trained across multiple PDE instances to handle a wider range of cardiac cell models. Although this would increase training costs, strategies such as query-point residuals or temporal-channel separation (Diab and Al-Kobaisi, 2025) could help maintain computational efficiency.

References

- Rubin R. Aliev and Alexander V. Panfilov. A simple two-variable model of cardiac excitation. *Chaos, Solitons & Fractals*, 7(3):293–301, 1996. doi: 10.1016/0960-0779(95)00089-5.
- Rafael Bischof and Michael A Kraus. Multi-objective loss balancing for physics-informed deep learning. *Computer Methods in Applied Mechanics and Engineering*, 439:117914, 2025.
- Hai-Yang Chen, Liang Xue, Li Liu, Gao-Feng Zou, Jiang-Xia Han, Yu-Bin Dong, Meng-Ze Cong, Yue-Tian Liu, and Seyed Mojtaba Hosseini-Nasab. Physics-informed graph neural network for predicting fluid flow in porous media. *Petroleum Science*, 2025. ISSN 1995-8226. doi: 10.1016/j.petsci.2025.06.007. URL <https://www.sciencedirect.com/science/article/pii/S199582262500216X>.
- Ching-En Chiu, Aditi Roy, Sarah Cechnicka, Ashvin Gupta, Arielev Pinto, Christoforos Galazis, Kim Christensen, Danilo Mandic, and Marta Varela. Physics-informed neural networks can accurately model cardiac electrophysiology in 3d geometries and fibrillatory conditions. In *International Workshop on Statistical Atlases and Computational Models of the Heart*, pages 98–109. Springer, 2024.
- W. Diab and M. Al-Kobaisi. Temporal neural operator for modeling time-dependent physical phenomena, 2025. URL <http://arxiv.org/abs/2504.20249>.
- Craig S. Henriquez. A brief history of tissue models for cardiac electrophysiology. *IEEE Transactions on Biomedical Engineering*, 61(5):1457–1465, 2014. doi: 10.1109/TBME.2014.2310515.
- Clara Herrero Martin, Alon Oved, Rasheda A. Chowdhury, Elisabeth Ullmann, Nicholas S. Peters, Anil A. Bharath, and Marta Varela. EP-PINNs: Cardiac electrophysiology characterisation using physics-informed neural networks. *Frontiers in Cardiovascular Medicine*, 8, 2022. doi: 10.3389/fcvm.2021.768419.
- A Ali Heydari, Craig A Thompson, and Asif Mehmood. SoftAdapt: Techniques for adaptive loss weighting of neural networks with multi-part loss functions. *arXiv preprint arXiv:1912.12355*, 2019.
- Zhihao Jiang, Houssam Abbas, Kuk Jin Jang, Marco Beccani, Jackson Liang, Sanjay Dixit, and Rahul Mangharam. In-silico pre-clinical trials for implantable cardioverter defibrillators. In *2016 38th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, pages 169–172, 2016. doi: 10.1109/EMBC.2016.7590667.
- Evangelia Katsoulakis, Qi Wang, Huanmei Wu, Leili Shahriyari, Richard Fletcher, Jinwei Liu, Luke Achenie, Hongfang Liu, Pamela Jackson, Ying Xiao, Tanveer Syeda-Mahmood, Richard Tuli, and Jun Deng. Digital twins for health: a scoping review. 7. ISSN 2398-6352. doi: 10.1038/s41746-024-01073-0. URL <https://www.nature.com/articles/s41746-024-01073-0>.
- Jean Kossaifi, Nikola Kovachki, Zongyi Li, David Pitt, Miguel Liu-Schiaffini, Robert Joseph George, Boris Bonev, Kamyar Azizzadenesheli, Julius Berner, and Anima Anandkumar. A library for learning neural operators. 2024. doi: 10.48550/arXiv.2412.10354.

- Nikola B. Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew M. Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces. *CoRR*, abs/2108.08481, 2021.
- Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. *arXiv preprint arXiv:2010.08895*, 2020.
- Zongyi Li, Nikola Borislavov Kovachki, Chris Choy, Boyi Li, Jean Kossaifi, Shourya Prakash Otta, Mohammad Amin Nabian, Maximilian Stadler, Christian Hundt, Kamyar Azizzadenesheli, and Anima Anandkumar. Geometry-informed neural operator for large-scale 3d PDEs, 2023. URL <http://arxiv.org/abs/2309.00583>.
- Zongyi Li, Hongkai Zheng, Nikola Kovachki, David Jin, Haoxuan Chen, Burigede Liu, Kamyar Azizzadenesheli, and Anima Anandkumar. Physics-informed neural operator for learning partial differential equations. *ACM/IMS Journal of Data Science*, 1(3):1–27, 2024.
- Miguel Liu-Schiaffini, Clare E. Singer, Nikola Kovachki, Sze Chai Leung, Hyunji Jane Bae, Kamyar Azizzadenesheli, and Anima Anandkumar. Tipping Point Forecasting in Non-Stationary Dynamics on Function Spaces, January 2026. URL <http://arxiv.org/abs/2308.08794>. arXiv:2308.08794.
- Hannah Lydon, Milad Kazemi Mehrabadi, Martin Bishop, and Nicola Paoletti. SimICD: A closed-loop simulation framework for ICD therapy. In *47th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, 2025.
- Iulia Nazarov, Ihsane Olakorede, Ahmed Qureshi, Shaheim Ogbomo-Harmitt, and Oleg Aslanidi. Physics-informed fully connected and recurrent neural networks for cardiac electrophysiology modelling. In *2022 Computing in Cardiology (CinC)*, volume 498, pages 1–4, 2022. doi: 10.22489/CinC.2022.188.
- Alexander V. Panfilov. Spiral breakup in an array of coupled cells: The role of the intercellular conductance. *Physical Review Letters*, 88(11):118101, 2002. doi: 10.1103/PhysRevLett.88.118101.
- Gernot Plank, Axel Loewe, Aurel Neic, Christoph Augustin, Yung-Lin Huang, Matthias A.F. Gsell, Elias Karabelas, Mark Nothstein, Anton J. Prassl, Jorge Sánchez, Gunnar Seemann, and Edward J. Vigmond. The openCARP simulation environment for cardiac electrophysiology. *Computer Methods and Programs in Biomedicine*, 208:106223, 2021. doi: 10.1016/j.cmpb.2021.106223.
- Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational physics*, 378:686–707, 2019.
- Francisco Sahli Costabal, Yibo Yang, Paris Perdikaris, Daniel E. Hurtado, and Ellen Kuhl. Physics-informed neural networks for cardiac activation mapping. *Frontiers in Physics*, 8, 2020. doi: 10.3389/fphy.2020.00042.
- Natalia A. Trayanova, Aurore Lyon, Julie Shade, and Jordi Heijman. Computational modeling of cardiac electrophysiology and arrhythmogenesis: toward clinical translation. *Physiological Reviews*, 104(3):1265–1333, 2024. doi: 10.1152/physrev.00017.2023.

Yan Barbosa Werneck, Rodrigo Weber dos Santos, Bernardo Martins Rocha, and Rafael Sachetto Oliveira. Replacing the FitzHugh-nagumo electrophysiology model by physics-informed neural networks. In Jiří Míkyška, Clélia de Mulatier, Maciej Paszynski, Valeria V. Krzhizhanovskaya, Jack J. Dongarra, and Peter M.A. Sloot, editors, *Computational Science – ICCS 2023*, pages 699–713. Springer Nature Switzerland, 2023. doi: 10.1007/978-3-031-36021-3_67.

A. Zolotarev, S. Ip, K. Vydyula, B. Dhillon, C. Martín, S. Misghina, and C. Roney. opencarp-pinns: Towards faster prediction of cardiac signals propagation, 2025. URL <https://stacom.github.io/stacom2025/papers/>.

Appendix S1. Training Data Generation

As described in Section 3, the Aliev-Panfilov equations model the transmembrane voltage, V , the recovery current, W , and the time t in atomic units (AU). The scaling for V and t is described in Equation 10.

$$V[AU] = \frac{V[mV] + 80}{100}, \quad t[AU] = \frac{t[ms]}{12.9}. \quad (10)$$

All of the propagation scenarios described in Section 4 were simulated using the open-source cardiac modelling software package openCARP (Plank et al., 2021) on a 2D $10 \times 10 \text{ cm}^2$ tetrahedral mesh with a resolution of $250 \mu\text{m}$ (resulting in a grid of resolution 401×401). All of the simulations were performed with isotropic conditions with the monodomain conduction model (Henriquez, 2014), and propagation was modelled using the Aliev-Panfilov cell model (using the parameters described in table 2).

Table 2: Parameters used for the Aliev-Panfilov residual loss

Propagation Scenario	D	a	μ_1	μ_2	k	ϵ
Planar, Centrifugal, Spiral	0.55	0.15	0.2	0.3	8	0.002
Spiral-Break	0.55	0.099	0.1	0.3	8	0.002

To produce the planar and centrifugal waves, the mesh was stimulated from a single wall and a single corner, respectively. Both the spiral and spiral-break-up waves were simulated using a cross-stimulation approach (pacing from a single wall, and then stimulating from a perpendicular wall at a later time in order to induce re-entrance). The simulations were generated with a time horizon of 1000ms for the planar and centrifugal scenarios in order to capture a single action potential, whilst the spiral and spiral-breakup models were generated with a time horizon of 2500ms in order to capture the long-term evolution of the system.

During the construction of the datasets used in training, the voltage field data was normalised to be in AU. When calculating the residual loss, the time was scaled to be in AU. For all propagation scenarios, we used the first 1000 ms of the simulation to create the training and testing datasets. During simulations, we set $dt = 100 \mu\text{s}$ for the forward solver, which resulted in an average simulation time of 40s per 1000ms simulation when running the simulation on our GPU server using 12 processing units.

For the parameter variation experiments described in Section S2.4.1, whilst we were not able to scale the parameter of D directly in the simulator, we were able to scale the isotropic conductivity, g_m . To relate this scaling to the value of D in the physics loss calculation, we equated the Aliev-Panfilov and monodomain formulations to infer the value of D using the values of membrane capacitance and the cell surface to volume ratio used in the openCARP simulator.

Appendix S2. Additional Experiments

S2.1. Model Baseline Evaluations

As referenced in Section 4, we conducted a baseline evaluation of the PINO model described in Section 3 against both a PINN and a purely data-driven FNO, using a fixed mesh resolution for both training and evaluation. For these evaluations, each propagation scenario was considered separately and the PINN was trained using the first 80% of each trajectory and tested on the last 20% to match the training split for the operator models. The training budgets for all models were on the order of 10^7 parameters; each model was trained for 1000 epochs with the specific propagation type, and the best-performing model was saved based on the MSE score on the test set. The error metrics for each model are shown in Table 3, which compares the baseline results for the FNO and PINO for single frame P2P prediction (as also shown in Tables 1 and 4), to the PINN model.

Table 3: Performance comparison of RMSE scores across models for each propagation scenario.

Model	Planar	Centrifugal	Spiral	Breakup
PINN	0.1114	0.1436	0.4593	0.3011
FNO	0.3766	0.0466	0.0248	0.0514
PINO	0.0095	0.0178	0.0047	0.0197

Compared to the operator models, the PINN’s performance was generally worse than the operator models when trained with an equivalent training budget. It is likely that the PINN model required more extensive training to fully learn the system dynamics, particularly for the more complex propagation scenarios – highlighting the operator model’s capability to learn system dynamics with a more computationally efficient training budget than neural network-based approaches.

S2.2. Long Horizon Predictions

As described in Section 4.1, we evaluated the performance of the spiral and spiral break-up models on long time horizon trajectories, extending the time horizon of the training trajectory by a factor of approximately 3. The results tend to be in agreement with Table 1 in that the PINO model is generally equivalent to or better than the FNO model. As expected, given the greater challenge of predicting over a longer time horizon, the RMSE is generally higher than in the baseline results, particularly for the spiral break-up model. This is primarily due to the difficulty of predicting chaotic systems over long time horizons, and suggests that enforcing the PDE as a soft constraint for learning wasn’t sufficient to capture the evolved dynamics of the system. Visualisations of the P2P results can be seen in the main body of the text in Figure 2.

S2.3. Training Epoch Variation

We experimented with the number of epochs that were used for the physics-informed training round to assess the effect on the model performance. We both reduced the number of epochs that were used from the PINO baseline¹¹ to 500 and increased it to 1500. The results can be found in Fig 6, which demonstrate that in the both the P2P and Roll-Out analysis, reducing the training to 500 epochs almost universally decreased the performance of the model whilst increasing the number of epochs to 1500 didn’t consistently improve model performance by any significant amount (with the exception of the spiral-break

11. Using the best performing training strategy for each propagation type.

Table 4: Baseline comparison for the FNO and all PINO models for a fixed mesh resolution, with the best performing model highlighted in bold.

(* indicates cases in which the model prediction collapsed.)

Scenario	Model	Single Frame		Multi Frame	
		RMSE (P2P)	RMSE (Roll-Out)	RMSE (P2P)	RMSE (Roll-Out)
Planar	FNO	0.3766	0.3819	0.3752	0.3794
	PINO-Fixed	0.0257	0.4735	0.0276	0.0442
	PINO-SoftAdapt	0.0120	1.9414*	0.0211	0.0204
	PINO-RelAdapt	0.0095	0.2561	0.0195	0.3776
Centrifugal	FNO	0.0466	0.3826	0.0629	0.3747
	PINO-Fixed	0.0415	0.3753	0.0072	0.0146
	PINO-SoftAdapt	0.0253	0.4661	0.0057	0.0112
	PINO-RelAdapt	0.0178	0.4027	0.0129	0.1418
Spiral	FNO	0.0248	0.4554	0.0507	0.1682
	PINO-Fixed	0.0060	0.0304	0.0113	0.0176
	PINO-SoftAdapt	0.0059	0.0295	0.0117	0.0179
	PINO-RelAdapt	0.0047	0.0047	0.0113	0.0173
Spiral-Break	FNO	0.0514	0.5939	0.1144	0.3132
	PINO-Fixed	0.0410	0.6004	0.0766	0.1925
	PINO-SoftAdapt	0.0432	0.5427	0.0758	0.1993
	PINO-RelAdapt	0.0197	0.3961	0.0832	0.3647

Table 5: Long-Horizon Baseline performances for a fixed mesh resolution

(* indicates cases in which the model prediction collapsed)

Scenario	Model	Single Frame		Multi Frame	
		RMSE (P2P)	RMSE (Roll-Out)	RMSE (P2P)	RMSE (Roll-Out)
Spiral	FNO	0.0270	0.5294	0.0525	0.2733
	PINO-Fixed	0.0100	0.0719	0.0200	0.0486
	PINO-SoftAdapt	0.0101	0.0923	0.0216	0.0535
	PINO-RelAdapt	0.0095	0.0742	0.0213	0.0393
Spiral-Break	FNO	0.0742	0.6321*	0.1810	0.3972*
	PINO-Fixed	0.0624	0.7444*	0.1447	0.4140
	PINO-SoftAdapt	0.0654	0.6109*	0.1298	0.4431
	PINO-RelAdapt	0.0577	0.5023	0.1490	0.4425

model). These results suggest that increasing the number of epochs can help the model to capture the more complex PDE dynamics of chaotic systems during training whilst for more stable propagation scenarios, increasing the training volume doesn't necessarily improve performance and can occasionally reduce the performance slightly, potentially as a result of over-fitting. In the interest of balancing model accuracy with computational training cost, we chose 1000 epochs as the baseline training budget.

S2.4. Generalisation Experiments

To evaluate the generalisation capabilities of the PINO approach, we compared the performance of models trained on a single propagation scenario with those trained on a mixture of propagation scenar-

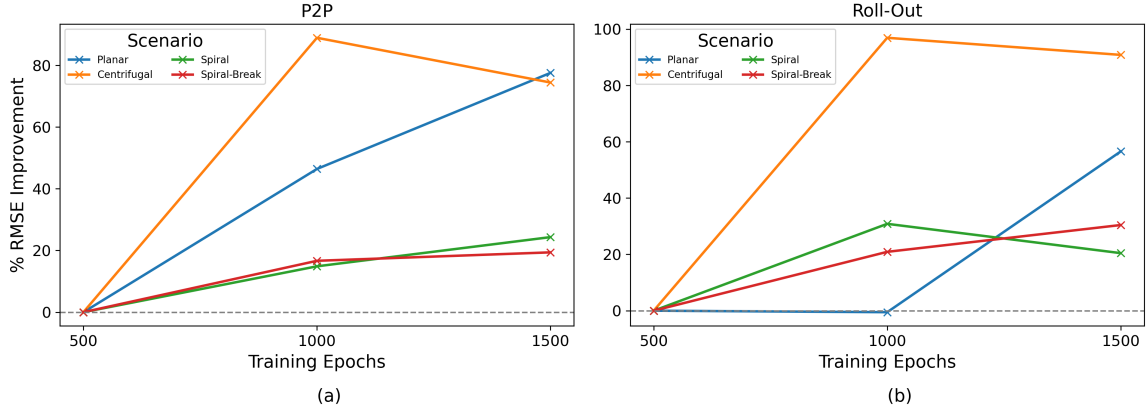


Figure 6: Comparison of Model Performance using Multi-Frame Training with training budget, showing the % RMSE improvement for (a) P2P evaluation, (b) Roll-Out evaluation.

ios and model parametrisations. When training these models, the number of epochs was increased to 10000 in order to allow the model more time to adapt to the wider range of sample types in the dataset.

S2.4.1. PARAMETER VARIATION

To evaluate the performance of the model on unseen parametrisations of the cell model, we varied the (isotropic) conductivity of the tissue, which is equivalent to a variation of the diffusion coefficient represented by the parameter D in Eq. 7, from the baseline value used in the main body of the text by factors in the range 0.5 to 1.5. Here, we considered the spiral model in particular, as the change in the conductivity resulted in significant variation in the movement of the spiral centre and the period of rotation. The training data was generated by varying the conductivity to create 11 different spiral trajectories, eight of which were selected at random to form the single frame input-output samples for training, using the same temporal splitting for the testing and training (training with the first 80% of the trajectory and testing on the last 20%) as described in 4. The remaining three trajectories were held out as evaluation sets to assess the model’s ability to adapt to out of distribution parametrisation.

During training, the parameters for each sample were passed to the model for use in the physics loss calculation; and at inference time the model was assessed using P2P evaluation on the full unseen trajectories. The model’s generalisation performance is shown in Table 6, where we compare the errors for the evaluation trajectories, representing out of distribution (OOD) parametrisation, to those calculated for the test set, representing in distribution (ID) parametrisation.

Table 6: Performance on spiral propagation under conductivity variation. Evaluation errors are computed over unseen parametrisations (OOD), while test errors correspond to held-out temporal segments of seen trajectories (ID). Δ and $\Delta\%$ denote the absolute and relative generalization gap (Eval – Test), respectively.

	Evaluation (Unseen D)	Test (Seen D)	$\Delta_{(Eval-Test)}$ ($\Delta\%$)
Mean RMSE	0.00368	0.00280	0.00088 (31.4%)

Although both the evaluation and test RMSE values remain low ($< 4 \times 10^{-3}$), there is a noticeable degradation in performance for the OOD trajectories. This suggests that, despite strong interpolation capabilities over seen conductivity regimes, the model exhibits reduced robustness when extrapolating to unseen parametrisations. One contributing factor may be the random selection of held-out trajectories, which could lead to a test set that is not sufficiently challenging or diverse. Additionally, the observed gap may indicate a degree of over-fitting to the parametrisations seen in training-testing dataset.

S2.4.2. ZERO-SHOT TRANSFER

As referenced in Section 4.3, we performed additional experiments to evaluate the model’s ability to perform predictions on unseen datasets. For these experiments, we trained models across multiple conductivity parametrisations for both the centrifugal and spiral models, this time using single frame samples taken from the full trajectories of each propagation type, with the test set consisting of samples from 2 of the same held out trajectories from the conductivity variations. At inference time, each model was evaluated on the remaining single unseen trajectory using P2P evaluation for both the propagation type that it was trained on and the unseen propagation type. The results of this evaluation can be found in Table 7¹².

Table 7: Performance comparison under parameter variation, including zero-shot transfer between propagation types. Error metrics are computed over full trajectories. Diagonal entries correspond to in-domain (ID) evaluation with respect to the propagation type; off-diagonal entries indicate zero-shot transfer between distinct propagation scenarios (OOD). Δ and $\Delta\%$ denote the absolute and relative generalization gap ($\text{OOD} - \text{ID}$), respectively.

Trained On	Centrifugal Eval (RMSE)	Spiral Eval (RMSE)	$\Delta_{(\text{OOD}-\text{ID})}$ ($\Delta\%$)
Centrifugal	0.0027	0.0679	0.0652 (2415%)
Spiral	0.0134	0.0037	0.0097 (262%)

Consistent with the previous parametrisation generalisation experiments, whilst the RMSE results for both the ID and OOD evaluation are low ($< 7 \times 10^{-2}$), there is a noticeable reduction in performance for the zero-shot prediction across the propagation type - especially for the centrifugal to spiral model. This result seems reasonable due to the spiral model being a more complex propagation scenario compared to the centrifugal propagation type.

As noted in Sections 4.3, the dominant source of error in the zero-shot setting lies in the amplitude of the predicted voltage rather than in the qualitative structure of the propagation patterns. This indicates that the model tends to successfully capture the underlying dynamics but struggles with accurate scaling for unseen propagation types, showing that there is scope to make improvements to the training procedure (such as enhanced normalisation, loss re-weighting, or amplitude-aware objectives) to improve zero-shot generalisation.

12. We note that this evaluation is OOD in terms of conductivity parametrisation for the evaluation that is ID for the propagation type